

Lekce 15: Čtení atributové tabulky pomocí kurzoru

Python pro GIS - Search Cursor

Vojtěch Barták, FŽP ČZU Praha

2025-12-01

Table of contents

1	Úvod	3
2	Čtení hodnot pomocí Search kurzoru	3
2.1	Typy kurzorů	3
2.2	Otevření kurzoru pomocí klauzule <code>with</code>	7
2.3	Speciální pole pro geometrii	7
2.4	Index kompaktnosti tvaru	10
2.5	Úlohy	11
3	Sumarizace tabulky pomocí kurzoru	11
3.1	Úlohy	13
4	Cyklus přes prvky vektorové vrstvy	14
4.1	Úlohy	17
5	Cyklus přes unikátní hodnoty pole	18
5.1	Úlohy	21
6	Cyklus přes unikátní hodnoty rastru	21
6.1	Atributová tabulka rastru	21
6.2	Úlohy	24
7	Cheatsheet	25
7.1	Základy práce s kurzory	25
7.1.1	Vytvoření Search kurzoru	25
7.1.2	Použití s klauzulí <code>with</code>	25

7.1.3	Procházení kurzoru	25
7.2	Speciální pole pro geometrii	26
7.2.1	Základní geometrická pole	26
7.2.2	Práce s geometrickými poli	26
7.3	Sumarizace dat	27
7.3.1	Součet hodnot	27
7.3.2	Průměr	27
7.3.3	Maximum/minimum	27
7.3.4	Podmíněný součet	27
7.4	Práce s unikátními hodnotami	28
7.4.1	Získání unikátních hodnot pole	28
7.4.2	Procházení unikátních hodnot	28
7.4.3	Sumarizace podle kategorií	28
7.5	Práce s rastry	29
7.5.1	Čtení atributové tabulky rastru	29
7.5.2	Výpočet ploch kategorií	29
7.5.3	Extrakce kategorií rastru	29
7.6	SQL dotazy (WHERE clause)	30
7.6.1	Základní operátory	30
7.6.2	Použití v kurzoru	30
7.7	Časté vzory použití	31
7.7.1	Export prvků podle kategorie	31
7.7.2	Vytvoření slovníku z tabulky	31
7.7.3	Hledání maxima s atributy	31
7.7.4	Procházení s indexem	32
7.8	Tipy a triky	32
7.8.1	Rychlé získání počtu prvků	32
7.8.2	Práce s None hodnotami	32
7.8.3	Kombinace více WHERE podmínek	33
7.8.4	Zpracování s chybovým hlášením	33

1 Úvod

V této lekci si ukážeme, jak lze procházet atributové tabulky (či jiné tabulky ve formátu **.dbf**) řádek po řádku a číst hodnoty z jednotlivých polí (sloupců). Po absolvování lekce budete umět:

- Otevřít kurzorem **dbf** tabulku a číst hodnoty v jednotlivých sloupcích
- Číst geometrické vlastnosti jednotlivých prvků z pole **SHAPE**
- Provádět vlastní sumarizace tabulek
- Zpracovávat vektorová data “prvek po prvek”
- Procházet v cyklu unikátní hodnoty z nějakého pole
- Procházet unikátní hodnoty diskrétního rastru

💡 Prozkoumejte nejprve data!

Jednotlivé procedury si budeme zkoušet na [datech](#) krajů (polygony) a velkých měst (body) ČR. Než začnete, nahrajte si data do ArcGIS Pro a prohlédněte si atributové tabulky!

2 Čtení hodnot pomocí Search kurzoru

Procházení tabulky se děje pomocí tzv. *kurzoru*, což je objekt podobný tomu, jaký vytváříme při otevírání textových souborů funkcí **open**: zprostředkovává spojení s daným souborem (tabulkou), takže po celou dobu, kdy tabulku procházíme, je uzamčena pro úpravy jinými aplikacemi. Z toho důvodu je po ukončení práce s tabulkou nutné vytvořený kurzor smazat (příkazem **del**), podobně jako po ukončení práce s textovým souborem jej uzavřeme metodou **close**.

2.1 Typy kurzorů

Kurzory v **arcpy** jsou trojího druhu:

- **Search Cursor** - určený pro pouhé čtení hodnot z tabulky,
- **Update Cursor** - určený pro čtení a zápis do existujících řádků tabulky,
- **Insert Cursor** - určený pro vkládání nových řádků do tabulky a mazání existujících řádků.

V této lekci si představíme pouze **SearchCursor**. Zbylé kurzory probereme v následující lekci.

Syntaxe použití Search kurzoru je následující:

```
tab = arcpy.da.SearchCursor(in_table, field_names, {where_clause}, ...)
```

V uvedeném kódu ukládáme kurzor do proměnné `tab`. Část `arcpy.da` odkazuje na skutečnost, že kurzory jsou v balíčku `arcpy` implementovány v samostatném modulu `da` (zkratka pro “Data Access”). Kurzor se vytváří stejnojmenným konstruktorem dané třídy (funkce `SearchCursor` v ukázce). Nejdůležitějšími parametry funkce jsou:

- **in_table** - tabulka, která se má otevřít (celá cesta jako řetězec, případně relativní cesta vzhledem k pracovnímu adresáři) - může se jednat o shapefile, `.dbf` tabulku, diskretní rastr (který má tabulku), vrstvu (pokud jsme ji vytvořili či pokud pracujeme v Python Window v ArcMap) apod.,
- **field_names** - seznam polí (sloupců), která chceme otevřít (pythonovský seznam řetězců),
- **where_clause** - SQL dotaz (nepovinně), kterým můžeme omezit otevíranou tabulku pouze na vybrané řádky.

Další parametry

Kurzor obsahuje ještě další parametry (znázorněné v ukázce třemi tečkami), ty zde však probírat nebudeme. Za zmínku nicméně stojí parametry `spatial_filter` a `spatial_relationship`, pomocí nichž lze kurzorem otevřít pouze prvky vybrané prostorovým výběrem (tj. za základě daného prostorového vztahu k dané jiné vrstvě).

Kurzor je možné přímo procházet `for` cyklem. Iterační proměnná přitom prochází jednotlivé řádky a nabývá vždy hodnoty n -tice stejné délky, jako je délka řádku (tj. počet otevřených polí). Ukažme si to na příkladu otevření atributů “KATEGORIE” a “NAZEV” vrstvy sídel:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"])
for r in tab:
    print(r)
del(tab)
```

Po spuštění skriptu obdržíme na konzoli následující výpis:

```
(u'Praha', 1)
(u'Brno', 2)
(u'Plze\u0148', 2)
(u'Ostrava', 2)
(u'Karlov Vary', 3)
(u'Zl\u00eddek', 3)
```

```
(u'Chomutov', 3)
(u'Most', 3)
(u'\xdast\xed nad Labem', 3)
(u'Kladno', 3)
(u'Hradec Kr\xe1lov\xed', 3)
(u'Karvin\xed', 3)
(u'Pardubice', 3)
(u'Fr\xfddek-M\xedstek', 3)
(u'Hav\xed\u0159ov', 3)
(u'Liberec', 3)
(u'\u010cesk\xed Bud\u011bjovice', 3)
(u'Olomouc', 3)
```

Jeliko\u017e je \u0159\u00e1dek jednoduchou n -tic\u00ed, je mo\u017en\u00e9 k jednotliv\u00fdm polo\u017dk\u00e1m (sloupc\u00fdm) \u0159\u00e1dku p\u0159istupovat pomocí index\u00fd:

```
# -*- coding: cp1250 -*-

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "KATEGORIE"])
for r in tab:
    print(f"M\u011bsto kategorie {r[1]} se jmenuje {r[0]}.")
del(tab)
```

V\u00fdpis vypad\u00e1 n\u00e1sledovn\u011b:

```
M\u011bsto kategorie 1 se jmenuje Praha.
M\u011bsto kategorie 2 se jmenuje Brno.
M\u011bsto kategorie 2 se jmenuje Plze\u0148.
M\u011bsto kategorie 2 se jmenuje Ostrava.
M\u011bsto kategorie 3 se jmenuje Karlovy Vary.
M\u011bsto kategorie 3 se jmenuje Zl\u00edn.
M\u011bsto kategorie 3 se jmenuje Chomutov.
M\u011bsto kategorie 3 se jmenuje Most.
M\u011bsto kategorie 3 se jmenuje \u00dast\u00ed nad Labem.
M\u011bsto kategorie 3 se jmenuje Kladno.
M\u011bsto kategorie 3 se jmenuje Hradec Kr\u00e1lov\u00e9.
M\u011bsto kategorie 3 se jmenuje Karvin\u00e1.
M\u011bsto kategorie 3 se jmenuje Pardubice.
M\u011bsto kategorie 3 se jmenuje Fr\u00fddek-M\u00edstek.
M\u011bsto kategorie 3 se jmenuje Hav\u00ed\u0159ov.
```

```
Město kategorie 3 se jmenuje Liberec.  
Město kategorie 3 se jmenuje České Budějovice.  
Město kategorie 3 se jmenuje Olomouc.
```

Všimněme si, že jsme při otevírání tabulky změnili pořadí sloupců oproti tomu, jak se tabulka zobrazuje v mapovém dokumentu ArcGIS Pro. Při otevírání tabulky pomocí kurzoru můžeme zvolit libovolné sloupce v libovolném pořadí, dokonce můžeme jeden sloupec otevřít v několika kopiích (což má zřídka nějaký dobrý důvod). Při čtení hodnot z řádku je přitom pořadí položek přesně takové, v jakém pořadí jsme sloupce seřadili při vytváření kurzoru.

V předešlé ukázce jsme nevyužili volitelný parametr *where_clause*, kterým můžeme omezit otvírané řádky. Pokud bychom např. chtěli otevřít pouze záznamy o městech první a druhé kategorie, vypadal by kód takto:

```
import arcpy  
arcpy.env.workspace = r"C:\my_path\..."  
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], '"KATEGORIE" < 3')  
for r in tab:  
    print(r[0])  
del(tab)
```

Výsledek:

```
Praha  
Brno  
Plzeň  
Ostrava
```

Opakované procházení kurzorem

Po průchodu tabulkou `for` cyklem zůstává kurzor na konci tabulky a nelze jej již znovu procházet. K tomu účelu je nutné buď kurzor znovu vytvořit, nebo jej vrátit na začátek tabulky metodou `reset`:

```

# první procházení
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"])
for r in tab:
    print(r[0])

# pokus o další procházení - NEFUNGUJE!
for r in tab:
    print(r[0])

# další procházení pomocí znovuvytvoření kurzoru
tab = arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"])
for r in tab:
    print(r[0])

# další procházení pomocí metody reset
tab.reset()
for r in tab:
    print(r[0])

```

2.2 Otevření kurzoru pomocí klauzule with

Nyní si ukážeme, jak se lze vyhnout nutnosti kurzor po použití mazat, a to díky klauzuli `with`, kterou znáte (v podobném kontextu) z lekce o textových souborech:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], '"KATEGORIE" < 3') as tab:
    for r in tab:
        print(r[0])

# Zde následují další příkazy, přičemž kurzor již je zavřený

```

2.3 Speciální pole pro geometrii

Kromě běžných atributových polí (jako "NAZEV", "POCET_OB" apod.) můžeme v kurzoru otevřít také speciální pole, která nám poskytují přístup ke geometrickým vlastnostem prvků. Tato pole mají speciální názvy začínající `SHAPE@`:

- `SHAPE@AREA` - plocha polygonu (v jednotkách souř. systému, typicky m²)

- SHAPE@LENGTH - délka linie nebo obvod polygonu (v jednotkách souř. systému, typicky m)
- SHAPE@XY - souřadnice bodu jako n -tice (x, y)
- SHAPE@TRUECENTROID - souřadnice těžiště polygonu jako n -tice (x, y)
- SHAPE@CENTROID - souřadnice těžiště polygonu, pokud těžiště leží uvnitř polygonu, jinak souřadnice popisku ("label point"), jako n -tice (x, y)

Ukažme si základní práci s těmito poli. Nejprve získáme plochy krajů:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Rozlohy krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha_m2 = r[1]
        plocha_km2 = plocha_m2 / 1000000 # převod na km²

        print(f"{nazev}: {round(plocha_km2, 2)} km²")
```

Můžeme kombinovat geometrická pole s běžnými atributy, například pro výpočet hustoty zalidnění:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Hustota zalidnění krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        obyvatel = r[1]
        plocha_km2 = r[2] / 1000000

        hustota = obyvatel / plocha_km2

        print(f"{nazev}: {round(hustota, 2)} obyvatel/km²")
```

Pro body můžeme získat jejich souřadnice pomocí SHAPE@XY:


```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Souřadnice velkých měst:")
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1] # rozbalení n-tice souřadnic

        print(f"{nazev}: X = {x}, Y = {y}")

```

Všimněte si, že `SHAPE@XY` vrací *n*-tici (x, y), kterou můžeme ihned rozbalit do dvou proměnných.

Podobně můžeme získat souřadnice těžišť polygonů:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Těžiště krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@TRUECENTROID"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]

        print(f"{nazev}: X = {x}, Y = {y}")

```

Pole `SHAPE@LENGTH` můžeme použít jak pro linie (kde vrací jejich délku), tak pro polygony (kde vrací obvod):

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Obvody krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        obvod_m = r[1]
        obvod_km = obvod_m / 1000

        print(f"{nazev}: {round(obvod_km, 1)} km")

```

Můžeme také kombinovat více geometrických polí najednou, například pro výpočet indexu kompaktnosti tvaru:

```
import arcpy
import math
arcpy.env.workspace = r"C:\my_path\..."

print("Index kompaktnosti krajů (nižší = kompaktnější):")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # km²
        obvod = r[2] / 1000 # km

        # Index kompaktnosti (kruh má nejnižší hodnotu 1)
        kompaktnost = (obvod ** 2) / (4 * math.pi * plocha)

        print(f"{nazev}: {round(kompaktnost, 2)}")
```

2.4 Index kompaktnosti tvaru

Index kompaktnosti porovnává tvar polygonu s kruhem (který má nejkompaktnější tvar ze všech útvarů). Vypočítá se jako:

$$\text{Kompaktnost} = \frac{O^2}{4\pi \cdot P}$$

kde O je obvod a P je plocha. Vzorec vychází z toho, že pro kruh platí $O = 2\pi r$ a $P = \pi r^2$. Po dosazení do vzorce dostaneme pro kruh hodnotu přesně 1. Čím více se tvar liší od kruhu (je protáhlejší, členitější), tím je hodnota vyšší. Například:

- Kruh: kompaktnost = 1,00
- Čtverec: kompaktnost 1,27
- Protáhlý obdélník: kompaktnost > 2,00

Konstantu π (pí) získáme v Pythonu importem modulu `math` a použitím `math.pi`.

Pokročilá práce s geometrií

Kromě zde uvedených polí existuje ještě `SHAPE@`, které vrací celý geometrický objekt s mnoha metodami pro pokročilé prostorové analýzy (buffery, průniky, obsahování atd.). Práci s těmito objekty si ukážeme v příští lekci.

2.5 Úlohy

Úloha 15.1. Otevřete tabulku krajů a vypište jejich názvy a počty obyvatel.

Úloha 15.2. Pomocí kurzoru najděte kraj s největší početní převahou žen nad muži.

Úloha 15.3. Pomocí geometrických polí vypočítejte a vypište hustotu zalidnění (obyvatel/km²) pro každý kraj. Najděte kraj s nejvyšší a nejnižší hustotou.

Úloha 15.4. Pro každé velké město vypište jeho název a souřadnice. Najděte město s nejvyšší souřadnicí Y (nejsevernější).

Úloha 15.5. Vypočítejte průměrnou plochu krajů v km². (Nápověda: projděte kurzorem všechny kraje, sečtěte plochy a vydělte počtem.)

Úloha 15.6. Pro jednotlivé plošky [krajinného pokryvu](#) vypočítejte index kompaktnosti tvaru. Jakého krajinného typu je ploška s nejkompaktnějším resp. nejméně kompaktním tvarem?

3 Sumarizace tabulky pomocí kurzoru

Jednou z velmi praktických aplikací kurzorů je provádění vlastních sumarizací tabulek. Zatímco nástroj **Statistics** nabízí předdefinovaný výběr statistik (suma, průměr, minimum, maximum apod.), pomocí kurzoru můžeme vypočítat jakoukoliv sumarizaci, kterou si sami naprogramujeme.

Princip je jednoduchý: procházíme tabulku kurzorem a průběžně akumulujeme hodnoty do proměnných. Ukažme si to na příkladu výpočtu celkového počtu obyvatel České republiky:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace sumy
celkem = 0

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        celkem += r[0]

print(f"Celkový počet obyvatel ČR: {celkem}")
```

Podobně můžeme vypočítat průměr, tentokrát s využitím čítače řádků:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace sumy a počítadla
suma = 0
pocet = 0

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1

prumer = suma / pocet
print(f"Průměrný počet obyvatel na kraj: {prumer:.0f}")

```

Zajímavějším příkladem je výpočet, jaké procento rozlohy ČR tvoří kraje s hustotou zalidnění nad 150 ob/km²:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Inicializace
plocha_husté = 0 # plocha hustě zalidněných krajů
plocha_celkem = 0 # celková plocha

# Procházení tabulky
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        obyvatel = r[1]
        plocha_m2 = r[2]
        plocha_km2 = plocha_m2 / 1000000

        hustota = obyvatel / plocha_km2

        plocha_celkem += plocha_km2

        if hustota > 150:
            plocha_husté += plocha_km2
            print(f"{nazev}: {hustota:.1f} ob/km2")

```

```
procento = (plocha_husté / plocha_celkem) * 100
print(f"\nHustě zalidněné kraje (>150 ob/km2) tvoří {procento:.1f}% rozlohy ČR")
```

Podobně můžeme např. spočítat počet krajů s nadprůměrným počtem obyvatel:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Nejprve vypočítáme průměr
suma = 0
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1
prumer = suma / pocet

# Nyní spočítáme kraje nad průměrem
nad_prumerem = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        if r[0] > prumer:
            nad_prumerem += 1

print(f"Počet krajů nad průměrem: {nad_prumerem} z {pocet}")
```

3.1 Úlohy

Úloha 15.7. Pomocí kurzoru vypočítejte celkový počet mužů a žen ve všech krajích ČR.

Úloha 15.8. Otevřete vrstvu okresů (*Okresy.shp*) a vypočítejte průměrnou hustotu zalidnění pro všechny okresy.

Úloha 15.9. Pro vrstvu Corine Land Cover (*CLC_CR.shp*) spočítejte celkovou plochu všech lesních ploch (kódy 311, 312, 313). Výsledek vypište v hektarech.

Úloha 15.10. Najděte tři okresy s nejvyšší hustotou zalidnění. Pro každý vypište název, počet obyvatel a hustotu. (Nápověda: použijte seznam pro ukládání dat a funkci `sorted()` pro seřazení.)

Úloha 15.11. Vypočítejte, kolik procent celkové rozlohy ČR zaujímají kraje s počtem obyvatel nad 1 milion.

Úloha 15.12. Pro vrstvu Corine Land Cover vytvořte CSV soubor s následujícími sloupci: CODE3 (kód kategorie), NAME3 (název kategorie), AREA_HA (plocha v hektarech), PERCENT (procento z celkové plochy). Soubor seřadte podle plochy sestupně. (Nápověda: nejprve projděte kurzorem všechny prvky a uložte data do seznamu slovníků, pak vypočítejte procenta a zapište do CSV.)

Úloha 15.13. Vytvořte CSV soubor se statistikami hustoty zalidnění pro všechny kraje ČR. Soubor by měl obsahovat sloupce: NAZEV, POCET_OB, ROZLOHA_KM2, HUSTOTA, KATEGORIE (kde kategorie je “velmi vysoká” pro hustotu >200, “vysoká” pro 100-200, “střední” pro 50-100, “nízká” pro <50).

Úloha 15.14. Pro každý okres spočítejte index kompaktnosti tvaru (podle vzorce z lekce) a najděte 5 nejkompaktnějších a 5 nejméně kompaktních okresů. Výsledky uložte do dvou samostatných CSV souborů s názvy `okresy_kompaktni.csv` a `okresy_nekompaktni.csv`.

4 Cyklus přes prvky vektorové vrstvy

Při práci s GIS daty často potřebujeme zpracovávat jednotlivé prvky samostatně - například provádět výpočty pro každý polygon zvlášť, vytvářet reporty po prvcích nebo exportovat prvky do samostatných souborů.

Kurzor nám umožňuje efektivně procházet všechny prvky vrstvy a číst jejich geometrické vlastnosti pomocí speciálních polí. Již jsme se setkali s polem `SHAPE@AREA`, které vrací plochu polygonu. Připomeňme si základní geometrická pole:

- `SHAPE@AREA` - plocha polygonu (v jednotkách souř. systému, typicky m^2)
- `SHAPE@LENGTH` - délka linie nebo obvod polygonu (v jednotkách souř. systému)
- `SHAPE@XY` - souřadnice bodu jako n -tice (x, y)
- `SHAPE@TRUECENTROID` - souřadnice těžiště polygonu jako n -tice (x, y)

Ukažme si, jak můžeme vypočítat kompaktnost jednotlivých krajů (poměr obvodu a plochy):

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Index kompaktnosti krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # převod na km2
        obvod = r[2] / 1000 # převod na km

        # Index kompaktnosti (nižší = kompaktnější tvar)
```

```

kompaktnost = (obvod ** 2) / plocha

print(f"{nazev}: {kompaktnost:.2f}")

```

Můžeme také pracovat se souřadnicemi bodů nebo těžišť:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Souřadnice velkých měst:")
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1] # rozbalení n-tice

        print(f"{nazev}: X = {x:.0f}, Y = {y:.0f}")

```

Pro polygony můžeme získat souřadnice jejich těžišť:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

print("Těžiště krajů:")
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@TRUECENTROID"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]

        print(f"{nazev}: X = {x:.0f}, Y = {y:.0f}")

```

Praktickým příkladem je vytvoření reportu o jednotlivých prvcích:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Vytvoření textového reportu
with open("report_kraje.txt", "w", encoding="utf-8") as f:
    f.write("REPORT O KRAJÍCH ČR\n")
    f.write("=" * 50 + "\n\n")

    with arcpy.da.SearchCursor("Kraje.shp",

```

```

["NAZEV", "POCET_OB", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        obyvatel = r[1]
        plocha = r[2] / 1000000 # km2
        obvod = r[3] / 1000 # km
        hustota = obyvatel / plocha

        f.write(f"Kraj: {nazev}\n")
        f.write(f" Počet obyvatel: {obyvatel:,}\n")
        f.write(f" Plocha: {plocha:.0f} km2\n")
        f.write(f" Obvod: {obvod:.0f} km\n")
        f.write(f" Hustota: {hustota:.1f} ob/km2\n")
        f.write("-" * 50 + "\n")

print("Report vytvořen: report_kraje.txt")

```

Můžeme také exportovat jednotlivé prvky podle kritérií do samostatných vrstev:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Export krajů podle velikosti
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # km2

        # Kategorizace podle plochy
        if plocha > 7000:
            kategorie = "velke"
        elif plocha > 4000:
            kategorie = "stredni"
        else:
            kategorie = "male"

        # Export do příslušné vrstvy
        where = f'"NAZEV" = \'{nazev}\''
        vystup = f"kraje_{kategorie}.shp"

        # Pokud soubor neexistuje, vytvoříme ho

```



```

if not arcpy.Exists(vystup):
    arcpy.Select_analysis("Kraje.shp", vystup, where)
else:
    # Jinak přidáme prvek k existujícím
    arcpy.Append_management("Kraje.shp", vystup, "NO_TEST", where)

print(f"{nazev} ({plocha:.0f} km²) -> {kategorie}")

```

Dalším příkladem je výpočet vzdáleností mezi body pomocí jejich souřadnic:

```

import arcpy
import math
arcpy.env.workspace = r"C:\my_path\..."

# Načtení souřadnic Prahy
with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@XY"], '"NAZEV" = \'Praha\'') as tab:
    praha_x, praha_y = next(tab)[0]

print("Vzdušné vzdálenosti od Prahy:")
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]

        if nazev != "Praha":
            # Euklidovská vzdálenost
            vzdalenost = math.sqrt((x - praha_x)**2 + (y - praha_y)**2) / 1000
            print(f"{nazev}: {vzdalenost:.1f} km")

```

Práce s geometrií v budoucích lekcích

V této lekci jsme se věnovali pouze základním geometrickým polím (SHAPE@AREA, SHAPE@LENGTH, SHAPE@XY). V následujících lekcích se naučíte pracovat s komplexními geometrickými objekty (SHAPE@), které umožňují provádět pokročilé prostorové analýzy jako buffery, průniky, obsahování a další operace.

4.1 Úlohy

Úloha 15.10. Pro každý kraj vypočítejte poměr počtu žen k počtu mužů a najděte kraj s největší “feminizací” populace.

Úloha 15.11. Vytvořte seznam krajů seřazených podle kompaktnosti (poměr obvodu a plochy). Vypište tři nejkompaktnější a tři nejméně kompaktní kraje.

Úloha 15.12. Pro každé velké město vypočítejte vzdálenost od těžiště nejbližšího kraje. (Nápověda: použijte dvojitý cyklus - vnější přes města, vnitřní přes kraje.)

5 Cyklus přes unikátní hodnoty pole

Častou úlohou při zpracování dat je procházet unikátní hodnoty nějakého kategoriálního pole a pro každou kategorii provádět specifickou operaci. Například můžeme chtít zpracovat všechny kraje podle krajů nebo města podle jejich kategorie.

V `arcpy` k tomu slouží jednoduchá funkce, která vrací seznam unikátních hodnot z daného pole:

```
arcpy.da.ListFieldValues(table, field)
```

Tato funkce vrací běžný pythonovský seznam, takže můžeme použít funkci `set()` k odstranění duplicit:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání unikátních kategorií měst
kategorie = set(arcpy.da.ListFieldValues("Sidla_body.shp", "KATEGORIE"))
print(f"Kategorie měst: {kategorie}")
```

Výstup:

Kategorie měst: {1, 2, 3}

Nyní můžeme kombinovat tuto funkci s kurzorem a procházet data podle kategorií:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání unikátních kategorií
kategorie = set(arcpy.da.ListFieldValues("Sidla_body.shp", "KATEGORIE"))

# Procházení každé kategorie
for kat in sorted(kategorie):
```

```

print(f"\n=== Města kategorie {kat} ===")

# SQL dotaz pro výběr měst dané kategorie
where = f'"KATEGORIE" = {kat}'

# Procházení měst v kategorii
with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(f" - {r[0]}")

```

Praktickým příkladem je sumarizace podle kategorií:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání unikátních kategorií
kategorie = set(arcpy.da.ListFieldValues("Sidla_body.shp", "KATEGORIE"))

print("Počty měst podle kategorií:")
for kat in sorted(kategorie):
    where = f'"KATEGORIE" = {kat}'
    pocet = 0

    with arcpy.da.SearchCursor("Sidla_body.shp", ["NAZEV"], where) as tab:
        for r in tab:
            pocet += 1

    print(f"Kategorie {kat}: {pocet} měst")

```

Můžeme také vytvářet samostatné výstupy pro každou kategorii:

```

import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Získání unikátních kategorií
kategorie = set(arcpy.da.ListFieldValues("Sidla_body.shp", "KATEGORIE"))

# Export měst podle kategorií
for kat in sorted(kategorie):
    where = f'"KATEGORIE" = {kat}'

```

```
vystup = f"Sidla_kategorie_{kat}.shp"

arcpy.Select_analysis("Sidla_body.shp", vystup, where)
print(f"Vytvořen soubor: {vystup}")
```

Pokročilejší příklad kombinuje procházení unikátních hodnot s geometrickými operacemi:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání seznamu krajů
kraje = set(arcpy.da.ListFieldValues("Kraje.shp", "NAZEV"))

print("Analýza měst podle krajů:")
for kraj in sorted(kraje):
    # Načtení geometrie kraje
    where_kraj = f'"NAZEV" = \'{kraj}\''
    with arcpy.da.SearchCursor("Kraje.shp", ["SHAPE@"], where_kraj) as tab:
        geom_kraj = next(tab)[0]

    # Počítání měst v kraji
    pocet = 0
    with arcpy.da.SearchCursor("Sidla_body.shp", ["SHAPE@"]) as tab:
        for r in tab:
            if geom_kraj.contains(r[0]):
                pocet += 1

    print(f"{kraj}: {pocet} velkých měst")
```

💡 Alternativa k ListFieldValues

Pokud potřebujete zpracovat pouze několik vybraných hodnot (nikoliv všechny unikátní), můžete hodnoty zadat ručně jako seznam:

```
vybrane_kategorie = [1, 2]

for kat in vybrane_kategorie:
    where = f'"KATEGORIE" = {kat}'
    # další zpracování...
```

5.1 Úlohy

Úloha 15.13. Vypište seznam všech krajů a pro každý kraj spočítejte jeho celkovou rozlohu v km².

Úloha 15.14. Pro každou kategorii měst spočítejte průměrnou vzdálenost mezi městy v dané kategorii. (Nápověda: budete potřebovat dvojitý cyklus přes města stejné kategorie.)

Úloha 15.15. Vytvořte slovník, který bude obsahovat pro každý kraj seznam velkých měst, která v něm leží (formát: {"Název kraje": ["Město1", "Město2", ...], ...}).

6 Cyklus přes unikátní hodnoty rastru

Podobně jako u vektorových dat často potřebujeme zpracovávat diskrétní rastry podle jejich unikátních hodnot. Typickým příkladem je landuse/landcover rastr, kde každá hodnota představuje jiný typ pokryvu.

Diskrétní rastr má v ArcGIS automaticky vytvořenou atributovou tabulku, kterou můžeme otevřít kurzorem stejně jako tabulku vektorové vrstvy. Ukažme si to na příkladu:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Otevření atributové tabulky rastru
with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet_pixelu = r[1]
        print(f"Hodnota {hodnota}: {pocet_pixelu} pixelů")
```

6.1 Atributová tabulka rastru

Atributová tabulka diskrétního rastru obsahuje standardně tato pole:

- VALUE - hodnota pixelu (kategorie)
- COUNT - počet pixelů s danou hodnotou
- případně další pole přidaná uživatelem

Pokud rastr nemá atributovou tabulku (např. je příliš velký nebo je kontinuální), musíte ji nejprve vytvořit nástrojem **BuildRasterAttributeTable**.

Získání unikátních hodnot z rastru je pak jednoduché:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání všech hodnot z rastru
hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))
print(f"Unikátní hodnoty v rastru: {sorted(hodnoty)}")
```

Nyní můžeme pro každou hodnotu provádět samostatné analýzy. Například můžeme vytvořit samostatný rastr pro každou kategorii:

```
import arcpy
from arcpy.sa import *
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Získání unikátních hodnot
hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))

# Vytvoření samostatného rastru pro každou kategorii
for hodnota in sorted(hodnoty):
    # Vytvoření binárního rastru (1 = daná kategorie, 0 = ostatní)
    vyraz = f"Con(Raster('landuse.tif') == {hodnota}, 1, 0)"
    vystup = Raster(vyraz)
    vystup.save(f"landuse_kategorie_{hodnota}.tif")

    print(f"Vytvořen rastr pro kategorii {hodnota}")
```

Můžeme také spočítat plochu jednotlivých kategorií (pokud známe velikost pixelu):

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."

# Získání velikosti pixelu
popis = arcpy.Describe("landuse.tif")
velikost_pixelu = popis.meanCellWidth * popis.meanCellHeight # v m2

print("Plochy jednotlivých kategorií:")
with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet = r[1]
```

```
plocha_ha = (pocet * velikost_pixelu) / 10000 # převod na ha

print(f"Kategorie {hodnota}: {plocha_ha:.2f} ha")
```

Praktickým příkladem je vytvoření vektorové vrstvy polygon pro každou kategorii rastru:

```
import arcpy
arcpy.env.workspace = r"C:\my_path\..."
arcpy.env.overwriteOutput = True

# Získání unikátních hodnot
hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))

# Konverze každé kategorie na polygony
for hodnota in sorted(hodnoty):
    # Extrakce dané kategorie
    vyraz = f'"VALUE" = {hodnota}'
    rastr_kategorie = arcpy.sa.ExtractByAttributes("landuse.tif", vyraz)

    # Konverze na polygony
    vystup = f"landuse_poly_{hodnota}.shp"
    arcpy.RasterToPolygon_conversion(rastr_kategorie, vystup, "NO_SIMPLIFY")

    print(f"Vytvořena polygonová vrstva: {vystup}")
```

Pokročilejší příklad kombinuje procházení kategorií rastru s prostorovými analýzami:

```
import arcpy
from arcpy.sa import *
arcpy.env.workspace = r"C:\my_path\..."

# Načtení referenční vrstvy (např. chráněná území)
chranena_uzemi = "chranena_uzemi.shp"

# Získání unikátních hodnot landuse
hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))

print("Zastoupení kategorií v chráněných územích:")
for hodnota in sorted(hodnoty):
    # Extrakce dané kategorie
    rastr_kategorie = arcpy.sa.ExtractByAttributes("landuse.tif", f'"VALUE" = {hodnota}')
```

```
# Oříznutí na chráněná území
rastr_clip = arcpy.sa.ExtractByMask(rastr_kategorie, chranena_uzemi)

# Spočítání pixelů
pocet = 0
with arcpy.da.SearchCursor(rastr_clip, ["COUNT"]) as tab:
    for r in tab:
        pocet += r[0]

if pocet > 0:
    print(f"Kategorie {hodnota}: {pocet} pixelů v CHÚ")
```

Práce s velkými rastry

Při zpracování velkých rastrů může být operace s kurzory pomalá. V takových případech zvažte:

- Použití nástroje `ZonalStatisticsAsTable` pro sumarizace
- Práci s pyramid levels pro rychlejší přístup
- Zpracování po částech (tiles)

6.2 Úlohy

Úloha 15.16. Pro diskrétní rastr landuse vypište všechny kategorie a jejich procentuální zastoupení vzhledem k celkové ploše.

Úloha 15.17. Najděte kategorii, která má největší průměrnou výšku (použijte výškový rastr DEM a nástroj `ZonalStatisticsAsTable` nebo vlastní analýzu s kurzory).

Úloha 15.18. Pro každou kategorii landuse vytvořte buffer 100 m a spočítejte, jaká část původní kategorie zůstává uvnitř tohoto bufferu. (Nápověda: nejprve převedte rastr na polygony, vytvořte buffer a použijte `Clip` nebo prostorové operace.)

7 Cheatsheet

7.1 Základy práce s kurzory

7.1.1 Vytvoření Search kurzoru

```
# Základní syntaxe
tab = arcpy.da.SearchCursor(in_table, field_names, where_clause)

# Příklad s konkrétními parametry
tab = arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"])

# S SQL dotazem
tab = arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], '"KATEGORIE" < 3')

# Uzavření kurzoru
del(tab)
```

7.1.2 Použití s klauzulí with

```
# Doporučený způsob - automatické uzavření
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"]) as tab:
    for r in tab:
        print(r[0])
```

7.1.3 Procházení kurzoru

```
# For cyklus (doporučeno)
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        print(f"{r[0]}: {r[1]} obyvatel")

# Metoda next() - posun na další řádek
tab = arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"])
prvni_radek = tab.next()
druhy_radek = tab.next()
del(tab)
```

7.2 Speciální pole pro geometrii

7.2.1 Základní geometrická pole

```
# Souřadnice bodu
["SHAPE@XY"] # vrací (x, y) tuple

# Plocha polygonu
["SHAPE@AREA"] # v jednotkách souř. systému (typicky m²)

# Délka linie nebo obvod polygonu
["SHAPE@LENGTH"] # v jednotkách souř. systému (typicky m)

# Těžiště polygonu
["SHAPE@TRUECENTROID"] # vrací (x, y) tuple
```

7.2.2 Práce s geometrickými poli

```
# Výpočet kompaktnosti z plochy a obvodu
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@AREA", "SHAPE@LENGTH"]) as tab:
    for r in tab:
        nazev = r[0]
        plocha = r[1] / 1000000 # převod na km²
        obvod = r[2] / 1000 # převod na km
        kompaktnost = (obvod ** 2) / plocha
        print(f"{nazev}: {kompaktnost:.2f}")

# Získání souřadnic bodů
with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV", "SHAPE@XY"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1] # rozbalení tuple
        print(f"{nazev}: X={x:.0f}, Y={y:.0f}")

# Těžiště polygonů
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "SHAPE@TRUECENTROID"]) as tab:
    for r in tab:
        nazev = r[0]
        x, y = r[1]
        print(f"Těžiště {nazev}: X={x:.0f}, Y={y:.0f}")
```

7.3 Sumarizace dat

7.3.1 Součet hodnot

```
celkem = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        celkem += r[0]
print(f"Celkem: {celkem}")
```

7.3.2 Průměr

```
suma = 0
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        suma += r[0]
        pocet += 1
prumer = suma / pocet
```

7.3.3 Maximum/minimum

```
maximum = float('-inf')
nazev_max = ""

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        if r[1] > maximum:
            maximum = r[1]
            nazev_max = r[0]

print(f"Největší: {nazev_max} ({maximum} ob.)")
```

7.3.4 Podmíněný součet

```
suma_velkych = 0
with arcpy.da.SearchCursor("Kraje.shp", ["POCET_OB"]) as tab:
    for r in tab:
        if r[0] > 1000000:
            suma_velkych += r[0]
```

7.4 Práce s unikátními hodnotami

7.4.1 Získání unikátních hodnot pole

```
# Z vektorové vrstvy
hodnoty = set(arcpy.da.ListFieldValues("Sidla.shp", "KATEGORIE"))

# Z rastru (atributová tabulka)
hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))
```

7.4.2 Procházení unikátních hodnot

```
kategorie = set(arcpy.da.ListFieldValues("Sidla.shp", "KATEGORIE"))

for kat in sorted(kategorie):
    where = f'"KATEGORIE" = {kat}'
    with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
        for r in tab:
            print(f"Kategorie {kat}: {r[0]}")
```

7.4.3 Sumarizace podle kategorií

```
kategorie = set(arcpy.da.ListFieldValues("Sidla.shp", "KATEGORIE"))
statistiky = {}

for kat in kategorie:
    where = f'"KATEGORIE" = {kat}'
    pocet = 0

    with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
```

```

        for r in tab:
            pocet += 1

    statistiky[kat] = pocet

print(statistiky)

```

7.5 Práce s rastry

7.5.1 Čtení atributové tabulky rastru

```

with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        hodnota = r[0]
        pocet_pixelu = r[1]
        print(f"Hodnota {hodnota}: {pocet_pixelu} px")

```

7.5.2 Výpočet ploch kategorií

```

popis = arcpy.Describe("landuse.tif")
velikost_px = popis.meanCellWidth * popis.meanCellHeight

with arcpy.da.SearchCursor("landuse.tif", ["VALUE", "COUNT"]) as tab:
    for r in tab:
        plocha_ha = (r[1] * velikost_px) / 10000
        print(f"Kategorie {r[0]}: {plocha_ha:.2f} ha")

```

7.5.3 Extrakce kategorií rastru

```

hodnoty = set(arcpy.da.ListFieldValues("landuse.tif", "VALUE"))

for hodnota in hodnoty:
    vyraz = f'"VALUE" = {hodnota}'
    rastr_kat = arcpy.sa.ExtractByAttributes("landuse.tif", vyraz)
    rastr_kat.save(f"kategorie_{hodnota}.tif")

```

7.6 SQL dotazy (WHERE clause)

7.6.1 Základní operátory

```
# Rovnost
'"KATEGORIE" = 1'
'"NAZEV" = \'Praha\''

# Nerovnost
'"KATEGORIE" <> 1'

# Porovnání
'"POCET_OB" > 1000000'
'"POCET_OB" <= 500000'

# Logické operátory
'"KATEGORIE" = 1 OR "KATEGORIE" = 2'
'"POCET_OB" > 100000 AND "POCET_OB" < 1000000'

# IN operator
'"KATEGORIE" IN (1, 2, 3)'

# LIKE operator (pro řetězce)
'"NAZEV" LIKE \'Pra%\'' # začíná na "Pra"
'"NAZEV" LIKE \'%ice\'' # končí na "ice"
```

7.6.2 Použití v kurzoru

```
# Jednoduchý dotaz
where = '"POCET_OB" > 1000000'
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])

# Složený dotaz
where = '"KATEGORIE" IN (1, 2) AND "POCET_OB" > 100000'
with arcpy.da.SearchCursor("Sidla.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])
```

7.7 Časté vzory použití

7.7.1 Export prvků podle kategorie

```
kategorie = set(arcpy.da.ListFieldValues("Sidla.shp", "KATEGORIE"))

for kat in kategorie:
    where = f'"KATEGORIE" = {kat}'
    vystup = f"Sidla_kat_{kat}.shp"
    arcpy.Select_analysis("Sidla.shp", vystup, where)
```

7.7.2 Vytvoření slovníku z tabulky

```
kraje_dict = {}

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
    for r in tab:
        kraje_dict[r[0]] = r[1]

print(kraje_dict)
# {'Praha': 1280508, 'Středočeský': 1338925, ...}
```

7.7.3 Hledání maxima s atributy

```
max_ob = float('-inf')
max_kraj = {}

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB", "SHAPE@AREA"]) as tab:
    for r in tab:
        if r[1] > max_ob:
            max_ob = r[1]
            max_kraj = {
                'nazev': r[0],
                'obyvatel': r[1],
                'plocha': r[2] / 1000000 # km2
            }
```

```
print(f"Největší kraj: {max_kraj['nazev']}")
print(f"Obyvatel: {max_kraj['obyvatel']}")
print(f"Plocha: {max_kraj['plocha']:.0f} km2")
```

7.7.4 Procházení s indexem

```
i = 0
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"]) as tab:
    for r in tab:
        i += 1
        print(f"{i}. {r[0]}")
```

7.8 Tipy a triky

7.8.1 Rychlé získání počtu prvků

```
pocet = 0
with arcpy.da.SearchCursor("Kraje.shp", ["OID@"]) as tab:
    for r in tab:
        pocet += 1
print(f"Počet prvků: {pocet}")

# Alternativa pomocí GetCount
pocet = int(arcpy.GetCount_management("Kraje.shp")[0])
```

7.8.2 Práce s None hodnotami

```
with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POZNAMKA"]) as tab:
    for r in tab:
        nazev = r[0]
        poznamka = r[1] if r[1] is not None else "Bez poznámky"
        print(f"{nazev}: {poznamka}")
```


7.8.3 Kombinace více WHERE podmínek

```
# Pomocí proměnných
min_ob = 500000
max_ob = 1500000
where = f'"POCET_OB" >= {min_ob} AND "POCET_OB" <= {max_ob}'

with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV"], where) as tab:
    for r in tab:
        print(r[0])
```

7.8.4 Zpracování s chybovým hlášením

```
try:
    with arcpy.da.SearchCursor("Kraje.shp", ["NAZEV", "POCET_OB"]) as tab:
        for r in tab:
            print(f"{r[0]}: {r[1]}")
except Exception as e:
    print(f"Chyba při zpracování: {e}")
```